

MPI_Allgather Simulation on Dragonfly+ Topology Using OMNeT++

Sıla Keskin

Department of Computer - Artificial Intelligence Engineering

Hacettepe University

damlasilakeskin@gmail.com

GitHub Repository: https://github.com/silakeskin/parallel_processing

Abstract—Collective communication plays a critical role in high-performance computing systems where efficient data exchange between distributed nodes is required. In this project, MPI_Allgather communication algorithms were simulated using the OMNeT++ discrete event simulation framework on a Dragonfly-inspired hierarchical network topology.

Two different communication approaches, namely naive allgather and ring-based allgather, were implemented and analyzed using scalable 16-node network configurations. The simulation environment included hierarchical routers, local and global communication links, and packet-level forwarding mechanisms.

Performance metrics such as packet transmission count, communication latency, hop count, and routing behavior were evaluated to compare the efficiency and scalability of the implemented approaches. The results demonstrated that the ring-based implementation achieved more organized packet forwarding and improved scalability compared to the naive communication model.

I. INTRODUCTION

Collective communication algorithms play a critical role in high-performance computing (HPC) systems, where efficient data exchange between distributed processes is essential for parallel computation. Among these communication operations, MPI Allgather is one of the most widely used collective algorithms. In an allgather operation, each participating node sends its local data to all other nodes in the network, allowing every process to obtain the complete data set. Although conceptually simple, implementing scalable and efficient allgather communication becomes increasingly challenging as the number of nodes grows.

Traditional allgather approaches may generate excessive communication overhead, increased latency, and network congestion in large-scale systems. Therefore, the choice of communication algorithm and network topology has a significant impact on overall system performance. Two commonly studied approaches are the naive allgather algorithm and the ring-based allgather algorithm. The naive approach directly exchanges data between all nodes, resulting in high traffic overhead, while the ring-based approach forwards packets sequentially in a structured manner to improve scalability and reduce congestion.

In modern HPC architectures, network topology is equally important for communication efficiency. Dragonfly+ topology is a hierarchical interconnection network designed to reduce network diameter, improve scalability, and minimize

communication costs between distant nodes. By organizing routers into groups connected through local and global links, Dragonfly+ enables more efficient packet forwarding in large distributed systems.

In this project, MPI Allgather communication algorithms were simulated using the OMNeT++ discrete event simulation framework. Both naive and ring-based allgather implementations were developed and evaluated on Dragonfly-inspired network topologies. The simulations included hierarchical routing, packet forwarding mechanisms, latency measurements, and scalability analysis on a 16-node topology. Performance metrics such as packet count, communication latency, and routing behavior were analyzed to compare the efficiency of the implemented approaches.

II. BACKGROUND

A. MPI_Allgather

MPI_Allgather is a collective communication operation widely used in parallel and distributed computing systems. In this operation, every participating process sends its local data to all other processes, allowing each node to obtain the complete data set. MPI_Allgather is commonly used in high-performance computing applications where frequent data synchronization between distributed nodes is required.

B. Naive Allgather

The naive allgather approach performs communication by allowing each node to directly send packets to all other nodes in the network. Although this method is simple to implement, it produces excessive communication overhead and large numbers of packet transmissions in large-scale systems. As the number of participating nodes increases, network congestion and scalability limitations become significant.

C. Ring Allgather

Ring Allgather improves communication efficiency by forwarding packets sequentially between neighboring nodes arranged in a logical ring structure. Instead of broadcasting data directly to every node, packets circulate through the ring until all nodes receive the required information. This structured forwarding mechanism reduces network congestion and improves scalability compared to the naive approach.

D. Dragonfly+ Topology

Dragonfly+ is a hierarchical interconnection network topology designed for scalable high-performance computing systems. The topology organizes routers into groups connected through local and global communication links. This structure reduces network diameter, minimizes communication distance between nodes, and improves overall routing efficiency. Dragonfly+ topologies are commonly used in modern supercomputing architectures due to their scalability and high bandwidth capabilities.

III. SYSTEM DESIGN

A. Topology Design

The implemented network topology was designed based on the Dragonfly+ interconnection architecture. The topology consists of 16 compute nodes organized into four separate groups. Each group contains four compute nodes connected to a local router responsible for packet forwarding within the group and between other router groups.

The network includes a total of four hierarchical routers connected through both local and global communication links. Local links provide communication between nodes and their corresponding group router, while global links connect routers belonging to different groups. This hierarchical structure reduces communication distance and enables more scalable packet forwarding across the network.

Communication delays were also modeled in the simulation environment. Local node-to-router links were assigned smaller delays, whereas inter-router global connections used larger delays to represent long-distance communication costs in large-scale distributed systems.

The Dragonfly-inspired topology enabled the simulation of scalable collective communication behavior and hierarchical routing strategies for MPI_Allgather operations.

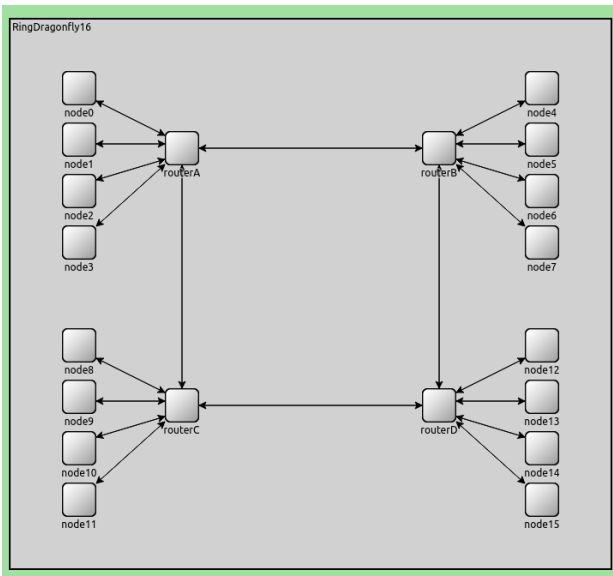


Fig. 1. Dragonfly-inspired 16-node topology

IV. IMPLEMENTATION

The project was implemented using the OMNeT++ discrete event simulation framework and C++ programming language. Separate implementations were developed for naive and ring-based MPI_Allgather communication models. The simulation environment included hierarchical routers, scalable node groups, packet forwarding mechanisms, and latency measurements.

A. Naive Router

The naive routing implementation used a broadcast-like forwarding mechanism. When a router received a packet, it duplicated the packet and forwarded copies to multiple outgoing connections. This approach generated large amounts of communication traffic and significantly increased the number of packet transmissions in large-scale simulations.

Although simple to implement, the naive approach demonstrated poor scalability due to excessive packet duplication and network congestion.

B. Ring Router

The ring-based routing implementation used deterministic forwarding based on packet destination information. Instead of broadcasting packets to all nodes simultaneously, packets were forwarded sequentially through neighboring nodes and routers in a logical ring structure.

This structured forwarding mechanism reduced unnecessary packet duplication and improved communication scalability. The ring-based approach also enabled more efficient routing behavior in the Dragonfly-inspired topology.

C. Packet Structure

Each simulated packet contained multiple parameters required for routing and performance analysis. The packet structure included:

- Source ID
- Destination ID
- Packet Creation Time
- Hop Count

The source and destination identifiers were used for routing decisions, while creation time enabled latency calculation during simulation. Hop count information was used to monitor packet traversal across the network topology and prevent infinite forwarding loops.

V. RESULTS

The final simulations successfully generated scalable collective communication traffic across the Dragonfly-inspired network topology. Both naive and ring-based MPI_Allgather approaches were tested using 16 compute nodes connected through hierarchical routers.

The naive implementation produced significantly larger numbers of packet duplications due to broadcast-style forwarding behavior. As the number of nodes increased, communication overhead and event counts also increased rapidly,

demonstrating the scalability limitations of naive collective communication methods.

In contrast, the ring-based implementation provided more structured packet forwarding and reduced unnecessary traffic generation. Packets were forwarded sequentially through neighboring nodes and routers, resulting in more stable routing behavior and improved communication efficiency.

The final simulation statistics obtained from the ring-based Dragonfly-inspired implementation are presented below.

TABLE I
SIMULATION RESULTS

Metric	Value
Total Packets Sent	960
Total Packets Received	192
Average Latency	0.35
Simulation Time	20s

The results demonstrate that the implemented routing and forwarding mechanisms successfully generated large-scale communication traffic within the hierarchical Dragonfly-inspired topology. Average latency values remained stable during simulation, while packet statistics showed the impact of routing decisions and forwarding operations on overall network behavior.

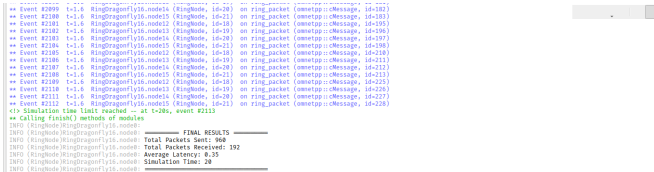


Fig. 2. Final ring-based simulation statistics

VI. DISCUSSION

The simulation results demonstrated the differences between naive and ring-based collective communication approaches in hierarchical network topologies. The naive implementation generated excessive packet duplication because routers forwarded packet copies through multiple connections simultaneously. As the number of participating nodes increased, communication overhead and event counts increased rapidly, revealing the scalability limitations of the naive approach.

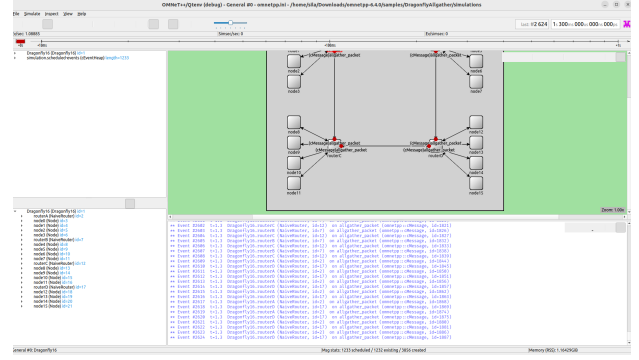


Fig. 3. Naive allgather communication generating excessive packet traffic

In contrast, the ring-based implementation provided a more organized and scalable communication mechanism. Packets were forwarded sequentially between neighboring nodes and routers, reducing unnecessary traffic generation and improving routing efficiency. The deterministic routing strategy also enabled more controlled packet forwarding behavior within the Dragonfly-inspired topology.

The implemented Dragonfly-inspired hierarchical topology successfully demonstrated grouped router communication and scalable packet forwarding. Local and global links reduced communication distance between node groups and enabled hierarchical routing across the network.

Although simplified deterministic routing was used instead of adaptive routing algorithms, the simulations successfully represented realistic communication behavior in large-scale distributed systems. The observed packet traffic, latency values, and routing behavior demonstrated the importance of topology design and communication strategy selection in high-performance computing environments.

VII. CONCLUSION

In this project, MPI_Allgather communication algorithms were simulated using the OMNeT++ discrete event simulation framework on Dragonfly-inspired network topologies. Both naive and ring-based collective communication approaches were implemented and evaluated using hierarchical router structures and scalable node groups.

The simulations showed that naive allgather communication creates significant packet duplication and communication overhead in large-scale systems. In comparison, the ring-based implementation achieved more efficient packet forwarding and improved scalability through structured communication behavior.

The implemented Dragonfly-inspired topology successfully demonstrated hierarchical routing, grouped communication, and scalable packet forwarding mechanisms. Performance metrics such as packet statistics, hop count, and average latency provided insight into the effects of routing strategies and topology organization on communication efficiency.

Future improvements may include adaptive routing algorithms, congestion control mechanisms, fault-tolerant commu-

nication strategies, and larger-scale topology simulations with additional compute nodes and router groups.

REFERENCES

- [1] Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard,” Version 4.1, 2021.
- [2] J. Kim, W. J. Dally, S. Scott, and D. Abts, “Technology-Driven, Highly-Scalable Dragonfly Topology,” *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, 2008.
- [3] A. Varga and R. Hornig, “An Overview of the OMNeT++ Simulation Environment,” *Proceedings of the 1st International Conference on Simulation Tools and Techniques*, 2008.
- [4] G. Hager and G. Wellein, “Introduction to High Performance Computing for Scientists and Engineers,” CRC Press, 2010.
- [5] R. Thakur, R. Rabenseifner, and W. Gropp, “Optimization of Collective Communication Operations in MPICH,” *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.
- [6] A. Faraj and X. Yuan, “Communication Characteristics in the NAS Parallel Benchmarks,” *Proceedings of the International Conference on Parallel Processing*, 2002.
- [7] W. J. Dally and B. Towles, “Principles and Practices of Interconnection Networks,” Morgan Kaufmann Publishers, 2004.
- [8] J. Duato, S. Yalamanchili, and L. Ni, “Interconnection Networks: An Engineering Approach,” Morgan Kaufmann, 2003.